

Как LLM меняет процесс разработки



Владимир Шилун
Старший frontend-разработчик
Drum Team

Сегодня в программе

01

Что тормозит разработку

Не скорость написания кода, а передача задачи между людьми и переключения между системами: рутина съедает почти все время

02

Как перестроился процесс

Порядок шагов переехал в инструкцию для LLM: файлы, цепочки, дежурства

03

Вопросы

Гейты и ответственность, чем это решение отличается от скриптов, что нужно для масштабирования

Запись, Q&A и подарки

Запись будет

Выложим на Rutube, YouTube и VK Видео

Вопросы — в чат трансляции

Разберем их в третьей части вебинара, пишите в общий чат

Время — около 55 минут

Разбор болей, решения и Q&A

В конце забираете артефакт

Шаблон описания действия + чек-лист внедрения, чтобы собрать такой слой у себя

БОЛИ РАЗРАБОТКИ_

Что на самом деле тормозит разработку

Как рутина съедает почти все время разработчика

Задача, которую не закрыть одному

01 · РУТИНА

Скорость написания кода — не главный тормоз. Рутинная съедает большую часть времени

02 · ПРОБЛЕМА РЕШАЕМА

В своей практике мы сталкиваемся с типовыми проблемами разработки, но смогли найти решение

03 · У ВАС ТОЖЕ ПОЛУЧИТСЯ

В конце вебинара — инструкция, как построить решение у себя

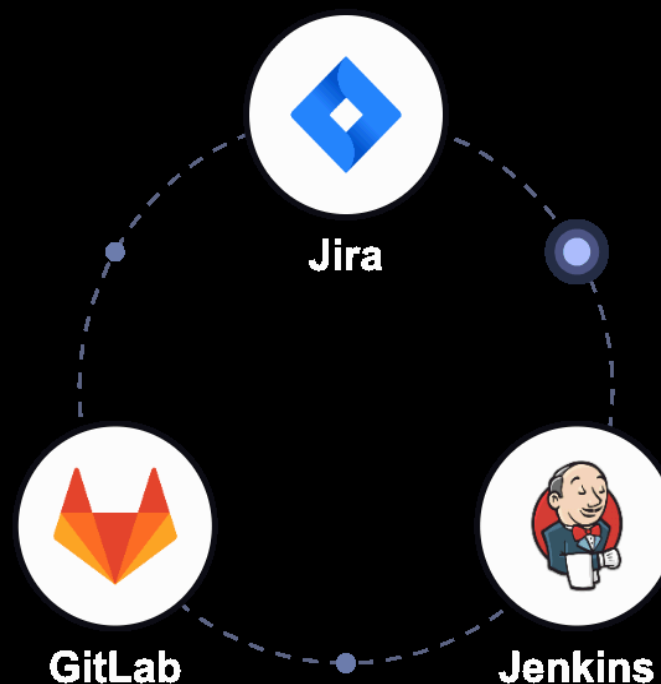
Как было раньше

Многошаговые операции

Мерж, сборки, комментарий в задачу, возврат тестировщику — десяток мелких действий на одну операцию

Одна задача — три системы: Jira, GitLab, Jenkins

На каждом шаге переключение: проверить апрувы, запустить нужную джобу, вставить ссылку на сборку в задачу



Как мы использовали LLM для борьбы с рутинной

ЧТО ДЕЙСТВИТЕЛЬНО УСКОРИЛОСЬ

Скорость написания кода: я перестал писать код руками — я его генерирую

Поиск по документации и по чужому коду. Погружение в задачу: почему упало, где искать

Это настоящий выигрыш — но он весь внутри того отрезка, где разработчик все делает сам

ЧТО ОСТАЛОСЬ КАК БЫЛО

Описать в задаче для тестировщика, что сделано и почему

Описать MR технически — для разработчиков

Мерж, сборки, возврат задачи, списание времени.

Переключения между трекером, репозиторием и сборкой

РЕШЕНИЕ_

Как перестроился процесс

Дополнительный слой между разработчиком и LLM

Три уровня

Отталкиваясь от нашей рутины — собрали архитектуру решения

ВЕРХНИЙ УРОВЕНЬ

Playbooks

Цепочки действий: «замержи → собери → переведи задачу»

СРЕДНИЙ УРОВЕНЬ

Actions

Одно логическое действие = один файл. Собрать MR с чек-листом по роли, завести баг из ошибки мониторинга

НИЖНИЙ УРОВЕНЬ

МСП-серверы

Сырые вызовы к API систем: трекер, репозиторий, сборка, мониторинг

Одно действие — это один файл

name: tracker/comment

Назначение

Опубликовать комментарий в задаче.

Инструменты

- `mcp\_\_tracker\_\_add\_comment`

Шаги

1. Собери текст: суть первой строкой, факты, ссылки.
2. Покажи превью и дождись подтверждения — комментарий видят все.
3. Вызови `add\_comment(issue, text)`.

Проверка

Тул вернул id комментария → опубликован.

Не вернул → покажи ошибку, повторно не отправляй.

Индексы

Решение: двухуровневый индекс

Запрос «напиши в задачу, что баг пофикшен»

↓
playbooks/_index.md ← 3 строки: не нашел нужного
↓
actions/_index.md ← 20 строк: по одной на действие. Выбрал jira/comment
↓
actions/jira/comment.md ← читает ЦЕЛИКОМ только этот файл
↓
догружает MCP-сервер → вызывает тул

Правила

- Индекс ****генерируется**** из frontmatter файлов. Руками не правят.
- В контекст попадает ~1 строка на действие вместо ~40.
- Промах на уровне playbooks → идет дальше на уровень actions
- Ничего не подошло → стоп, спроси. Не импровизируй.

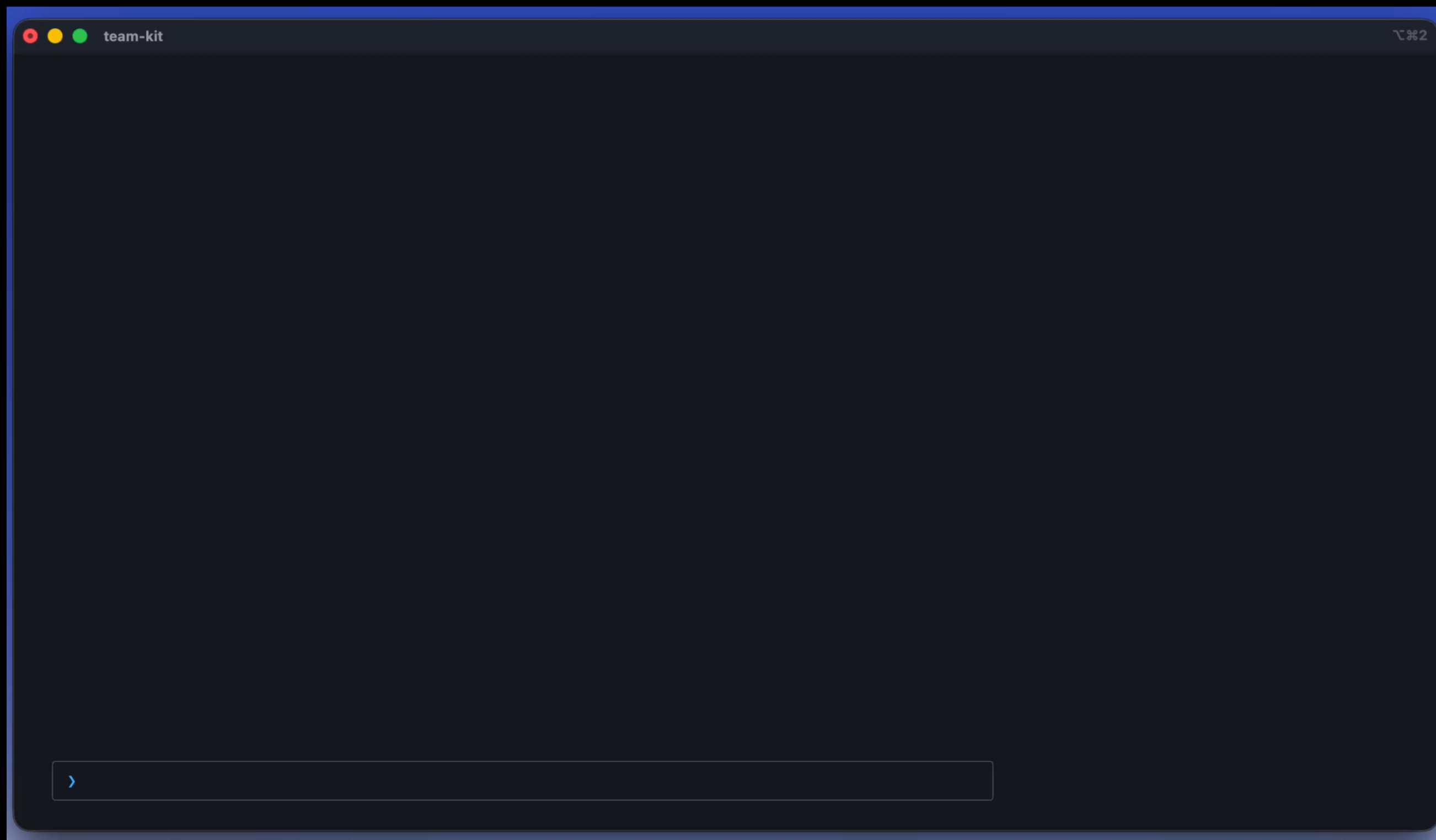
jira/

- read-issue — прочитать задачу и показать сводку
- comment — написать комментарий в Jira wiki markup

git/

- create-branch — создать локальную ветку от базовой и переключиться на неё
- commit — закоммитить с форматом сообщения (Jira-ключ из имени ветки, английский)

Одна фраза — три системы: прогон мержа



Одна фраза — три системы

«Замержи ZB-12345»

→ десять шагов в трёх системах одним запросом, в правильном порядке

Многошаговые операции — одной фразой

Нет прыжков между трекером, репозиторием и сборкой

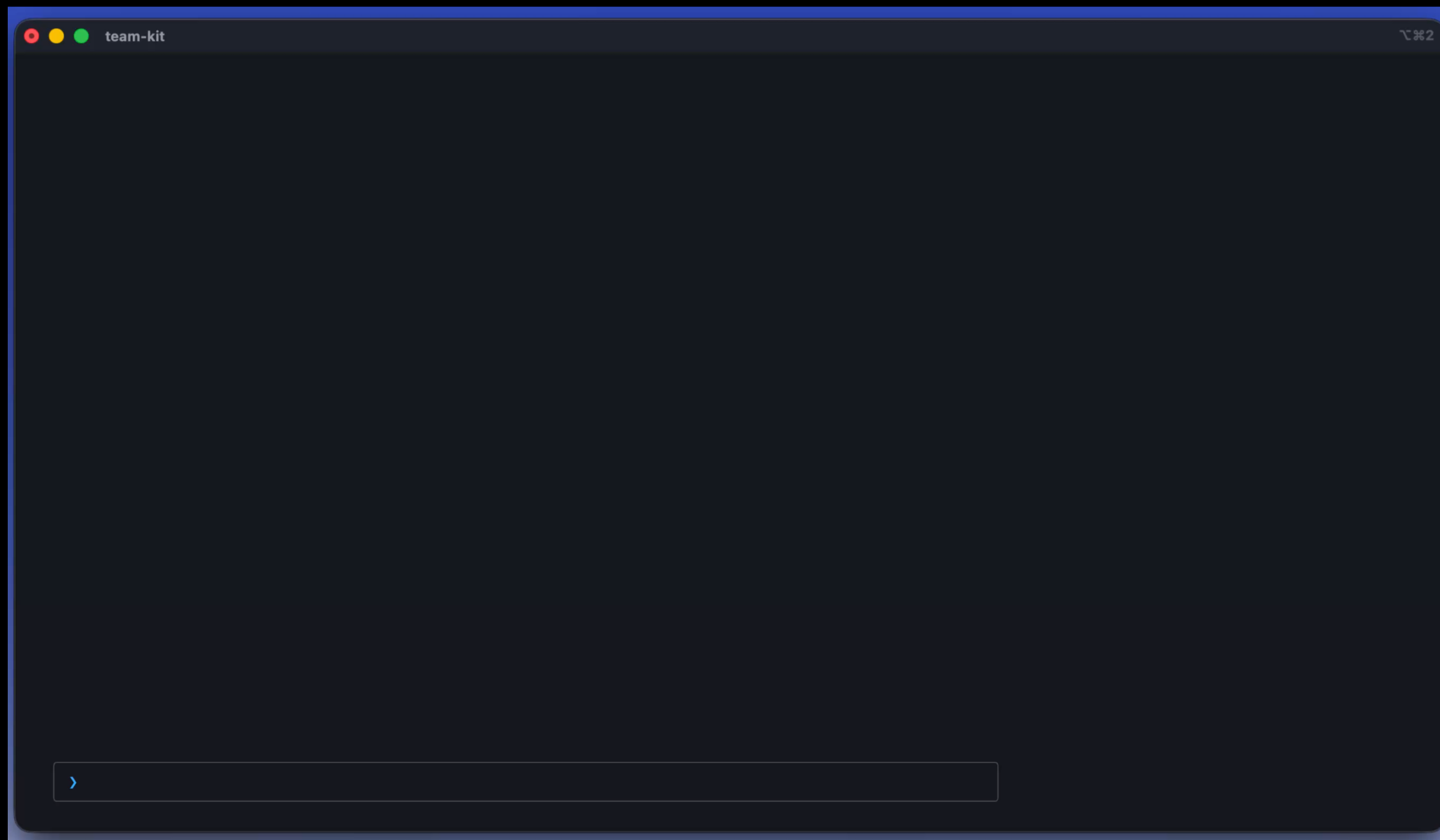
Порядок и проверки — в инструкции, а не в голове

Меньше ручных действий — меньше случайных ошибок

ЦЕПОЧКА МЕРЖА: 10 ШАГОВ, 3 СИСТЕМЫ, 2 ПОДТВЕРЖДЕНИЯ

Найти MR → апрувы (меньше двух — стоп) → обсуждения (открыты — стоп) → конфликты → подтверждение → мерж в main → сборка → черновик комментария → превью и подтверждение → публикация → задача в интеграцию → списать время

Дежурство целиком: без участия человека



Рутина дежурного — автоматически

01

Собрать ошибки

02

Отсеять шум

03

Распределить по
экспертизе

04

Отчет

Где остались проверки

Проверки не исчезли. Они переехали из памяти человека в обязательные шаги инструкции — и стали надежнее, чем «разработчик помнит, что надо»

Агенту можно запретить любое действие, где хотим оставить контроль

Хуки: предохранители, чтобы Claude не сделал лишнего

Все, что выходит во внешнее окружение, не может уйти без разрешения

«А ваш общий стандарт не ломает мой процесс?»

local — человек

Личные настройки

Playbook, actions, duty

team — команда

Свои проекты, процессы и сценарии

Playbook, actions, duty

common — компания

Единые правила и стандарты для всех

Playbook, actions, duty

Резолв local → team → common: побеждает более специфичный слой

Метрики успеха изменений процесса

ЗАКРЫТИЕ ЗАДАЧИ

Цепочка из 9 шагов в 3 системах

Было: ~10 переключений

Стало: одна фраза. человек подтверждает один шаг

ВОПРОС ИЗ ЧАТА

Разобрать вопрос из переписки

Было: руками донести контекст — задача, ветка, стенд, логи

Стало: копипаст контекста → одна ссылка

ОНБОРДИНГ

Что может новый человек сам

Было: мерж — на третий день

Стало: мерж по регламенту в первый день

ДЕЖУРСТВО ПО ОШИБКАМ

Разбор новых групп ошибок

Было: ~ 50–60 минут

Стало: 10–15 минут

Что не сработало

ЧТО ШЛО НЕ ТАК, КАК ОЖИДАЛИ

При наличии трех слоев Claude игнорировал playbooks, actions, а шел сразу в MCP-сервер выполнять

Не каждая инструкция может описать все, что на самом деле хочет человек

При расширении на команду — полетели merge-requests, которые хотели дополнять свои инструкции, свои playbooks, свои actions

Текущая проблема: разработчики добавляют слишком много своих скриптов, это делает систему нестабильной. Потенциальное решение — расслоение архитектуры

Что нужно, чтобы собрать такой слой у себя

Любая агентная среда

MCP-сервер

Инструкции — обычный текст

ВАШИ ВОПРОСЫ_

Q&A

Разбираем вопросы



НЕ ЗАБУДЬТЕ СКАЧАТЬ_

Шаблон описания действия для внедрения у себя

Восемь файлов на примере команды разработки.
По одному файлу на каждый слой архитектуры.

Скачивайте чек-лист для реализации решения под свои задачи

Обязательные поля, секции и место проверки результата — сразу в готовом виде

Запись эфира выложим на Rutube, YouTube и VK Видео



СООБЩЕСТВО

Вступайте в Just AI Dev Community

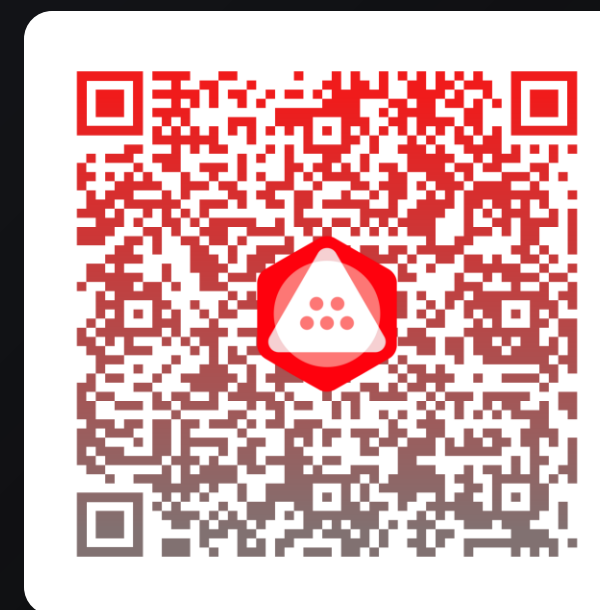
Сообщество для разработчиков: практика, разборы и анонсы эфиров



CAILA

Доступ к 350+ LLM

Один эндпоинт для всех моделей — тот самый, на котором работает наш слой



Следующий вебинар — в октябре

Продолжим говорить об эффективном использовании LLM в инженерной практике — обсудим вопросы безопасности и неочевидные риски при работе с моделями

Предлагаем вам посмотреть наши [прошедшие вебинары](#)

БИЛЕТЫ НА CONVERSATIONS 2 ДЕКАБРЯ УЖЕ В ПРОДАЖЕ

Blind bird

-30% на билеты до
первых спикеров

КУПИТЬ БИЛЕТ



Билеты на сайте
conversations-ai.com



Быть в курсе
t.me/conversations_ai

“

CONVERSATIONS

БЛАГОДАРИМ ВАС_

СПАСИБО ЗА ВНИМАНИЕ

До новых встреч!

